FULL-STACK AI TRACK · DEEP DIVE DD6

Build an AI feature that cannot make things up

Build a Grounded AI Feature Lab

Participant Handbook · read before Wednesday

When	Wednesday 28 October 2026, 08:00 to 11:00 IST
Format	A hands-on **milestone lab**, mostly on your own: success criteria, hints and checkpoints, not a follow-along. No prompts are handed out. The facilitator checks in twice, after milestone 4 and after milestone 8.
Where	Virtual (Zoom or Teams), working on the Linux lab VM with VS Code and Claude Code
Before you come	About 60 to 75 minutes of reading, plus a 15-minute warm-up. **Nothing to install.** The lab kit is handed out at the start of the session.
Keep it open	Part 5 (the terminal handbook) and the one-page cheat sheet at the back sit next to your terminal on the day

PwC Emerging Business · upGrad Enterprise · Full-Stack AI Track · Delivery material

Contents

1. Start here
2. Part 1 · The session at a glance
3. Part 2 · Before you arrive
4. Part 3 · The ideas
5. Part 4 · How the lab works
6. Part 5 · The terminal handbook
7. Part 6 · The kit, mapped
8. Part 7 · Glossary
9. Part 8 · Answers, reading and the cheat sheet
10. One-page cheat sheet

If you only have 30 minutes: read **Part 1**, the five **"In plain English"** boxes in Part 3, and Part 5, sections 5.1 to 5.5.

Start here

What this handbook is for

On Wednesday 28 October you will spend three hours building an AI feature that answers only from the documents it is given, and shows where each answer came from. It is a hands-on session, and it is different from a follow-along: **nobody will give you prompts to paste, and nobody will stop the room after every step.** You will get goals, success criteria and a clock, and you will work out how to get there yourself, with Claude Code, by hand, or both.

That only works if you arrive knowing three things, and this handbook gives you all three:

1. **The ideas** (Part 3): why a bot makes things up, what "grounded" means, and how chunks, embeddings and a database fit together.
2. **The method** (Part 4): how a milestone works, how the hints and the two check-ins work, and how to brief Claude Code well.
3. **The commands** (Part 5): every command you will type, what it does, why we run it, and how to read what it prints.

How long it takes. About 60 to 75 minutes to read properly, plus 15 minutes for the warm-up and the self-check in Part 2. If you have only 30 minutes, read Part 1 (the shape of the day), the five "In plain English" boxes in Part 3, and Part 5.1 to 5.5.

What you do not need to do. You do not need to install anything beforehand. The lab kit is handed out at the start of the day, and a couple of minutes later it is running. This handbook needs nothing but a reader.

Keep it open on the day. Part 5 (the terminal handbook) and the one-page cheat sheet at the back are made to sit next to your terminal while you work.

What you will be able to do afterwards

By the end of the session you will be able to:

- explain, in plain words, why an AI feature invents answers, and what "grounded" means;
- split a structured document into chunks without separating a fact from its context;
- store text and its embeddings in a real vector database, and keep it up to date;
- find the chunks that best match a question, and say why the best match is not always the right one;
- write a prompt that makes the model answer only from what it was given, cite its source, and say "I don't know" when it must;
- test a feature with question and answer pairs that can really fail, and tell a retrieval failure from a generation failure;
- stop a stale document from shipping in silence;
- brief an AI coding assistant so the result is checkable, and review what it gives you.

The last two are habits more than facts. They are the point of doing this as a lab.

Where this fits

This is DD6 of 33 sessions in the Full-Stack AI track. Earlier sessions gave you the pieces this one leans on. On **Monday** (the masterclass on LLM APIs and RAG basics) you turned a vague "AI FAQ bot" into a feature definition, and wrote an evaluation set with questions that are in scope, at the edge, and out of scope: today you build the thing that set was written for, and you use your questions to test it. **DD4** showed a vector search in a real Postgres. **DD5** (the auth lab) ended with a promise that today's retrieval feature would reuse its idea of one narrow scope per tool: today's lab is the pipeline, not the endpoint in front of it, and the homework asks you to think about that endpoint. The pipeline you build today is the starting point for the **capstone**, and **DD10** (Wednesday 11 November) turns today's hand-made question pairs into a proper evaluation harness.

Part 1 · The session at a glance

The scenario

A phone company has an AI FAQ bot. A customer asked it for the price of the **Unlimited Plus** plan, and it answered with a price, fluently and with confidence. That price is not in the plan sheet. It does not exist anywhere in the company's documents.

Nothing has gone wrong yet except that one customer was told something false. Your job over the morning is to reproduce that failure, take the bot apart, build a pipeline that ties every answer to a source you can check, prove it works with tests that can fail, and make sure it cannot quietly go out of date.

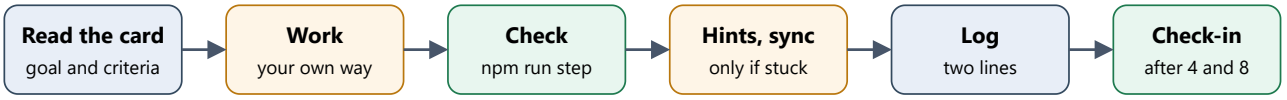
The agenda (Wednesday 28 October, 08:00 to 11:00 IST)

Time	What happens
08:00	**Kick-off.** The scenario, how the morning works, the ground rules
08:05	**Get the kit.** You receive a link, save a zip, and run four commands. It installs while we carry on
08:09	**Concept sprint.** Architecture, grounding, chunking, embeddings, citations, freshness. A question first, then the answer
08:22	**Live demo.** A rerank step that fixes a wrong ranking, as a worked example of the method you will use
08:32	Break
08:39	**Lab briefing.** How the cards, the hints and the check-ins work
08:43	**Block A: milestones 1 to 4, on your own.** Reproduce the bug, chunk, ingest, retrieve
09:28	**Check-in 1.** The facilitator walks through the steps, the tests and the solutions for milestones 1 to 4
09:43	Break
09:49	**Block B: milestones 5 to 8, on your own.** The grounded prompt, the whole answer, fixture tests, a staleness check
10:28	**Check-in 2.** The same, for milestones 5 to 8
10:44	**Wrap-up.** Reading the output, tools, a short exit quiz, the homework, the close
11:00	End

About two thirds of the session is the lab, and most of that is you working by yourself. The talking is kept short because this handbook did the explaining.

How the lab works

Each of the eight milestones is written like a ticket in a real engineering team. It tells you the **goal**, why it matters, what you are **given**, the **interface contract** (the exact file names, function names and return shapes that other code depends on), and the **success criteria**. It does **not** tell you how to do it.



then the next milestone, from known-good code

Every milestone follows the same loop. The room only stops twice.

- **Read the card.** The success criteria are your acceptance criteria. Read the card before you touch the keyboard.
- **Work.** You decide how. Briefing Claude Code is a good way; so is doing it by hand. What matters is the result.
- **Check.** `npm run step -- 4` tells you which success criteria are not met yet. You never have to ask anyone whether you are done.
- **Hints and sync, only if you are stuck.** Three hints per milestone, which open a few minutes after you start it. You take them yourself with a command (`npm run hint -- 4`). And `npm run checkpoint -- 4` gives you the reference code, so the next milestone starts from a known-good place. Your own work is saved first.
- **Log.** Two or three lines in your own evidence log: what you tried, what you learned.
- **Check-in.** After milestone 4 and after milestone 8 the facilitator walks through the steps, the tests and the solutions, and shows the reference brief (the "correct prompt") next to what you wrote.

What is expected of you, and what you get

You are expected to	You get
Read each card before you touch the keyboard	A written ticket for every milestone, with success criteria you can check yourself
Write your own briefs, review every change, and run the check	An automatic check per milestone that tells you which criteria are not met yet
Try before you take a hint, and say so when you take one early	Three hints per milestone, then the reference solution at the check-ins
Say in the chat when you are stuck (it is not a failure to be stuck)	Teaching assistants watching the chat, and a free sync whenever you need one
Never paste a real key or password anywhere	A kit where everything is made up, so you cannot leak anything by accident

How "done" is measured

You are done with a milestone when its check says **PASSED**: `npm run step -- 4` prints `STEP 4 PASSED`. You can check everything at once with `npm run progress`. Nobody grades your log. The session ends with a short exit quiz (five questions) that maps to the course's assessments, and a feedback form.

A milestone you reached by syncing from a checkpoint still counts as understood **if you can explain it**. The log has a line for that.

Part 2 · Before you arrive

Readiness checklist

Tick these off before Wednesday. None of them needs the lab kit.

☐	Check	How to check it
☐	I can reach the lab VM	Use the access details from your coordinator. If you cannot get in, tell them now, not on Wednesday morning.
☐	VS Code opens on the VM	Open it and look for a window with a file explorer on the left.
☐	**Claude Code is logged in**	It is the same login you used on Monday 5 October. Open the Claude Code panel in VS Code and send it one line ("hello"). If it answers, you are ready.
☐	I can open a terminal inside VS Code	Menu: Terminal, then New Terminal. You should see a prompt that ends in `$`.
☐	Node is new enough	In that terminal: `node --version`. It must print `v20` or higher.
☐	I can see the Zoom or Teams chat while the terminal is open	Two windows side by side, or a second screen. You will paste a line into the chat at the two check-ins.
☐	Notifications are off, and I have three uninterrupted hours	Including mail and chat.
☐	I have my questions from Monday's exercise, or I will write new ones	In the masterclass you drafted an evaluation set for the FAQ bot. Milestone 7 turns one into a test. If you lost it, that is fine.
☐	I have read Part 1, the "In plain English" boxes in Part 3, and Part 5	About 30 minutes if you are short of time.
☐	I have done the warm-up below	About 15 minutes.

You do not need: an API key, Docker, a database, or a Postgres install. The kit contains everything. The plan sheet, the prices and the roaming policy in it are made up, so nothing you do on the day can leak anything real.

Warm-up (about 15 minutes, nothing to install)

The aim is to make three things feel ordinary before the clock starts: the shell, what happens when you cut a document in the wrong place, and what "similar" means to a computer. All you need is a terminal with Node. Use any folder.

Exercise 1 · The shell, in five commands

```
pwd                   # print the folder you are in
ls -A                 # list what is in it, including hidden files (names starting with a dot)
echo "hello"          # print a line of text
NAME=$(echo hello)    # run the command in the brackets, and keep what it printed in a variable
echo "got: $NAME"     # use the variable
```

Then try these two, which show something you will rely on all morning: every command ends with an **exit code**, `0` for success and anything else for failure.

```
true;  echo "exit code: $?"      # prints 0
false; echo "exit code: $?"      # prints 1
```

Exercise 2 · Cut a price list in the wrong place

Make a small file that looks like the plan sheet you will meet in the lab:

```
cat > plans.txt <<'EOF'
## Basic plan
Price: $9.99 per month
- 2 GB of data

## Student plan
Price: $14.99 per month
- 20 GB of data
- Valid student ID required
EOF
```

First cut it into pieces of exactly 60 characters, which is what a simple "chunker" does:

```
node -e 'const t=require("fs").readFileSync("plans.txt","utf8"); for (let i=0;i<t.length;i+=60)
console.log(JSON.stringify(t.slice(i,i+60)))'
```

```
"## Basic plan\nPrice: $9.99 per month\n- 2 GB of data\n\n## Stud"
"ent plan\nPrice: $14.99 per month\n- 20 GB of data\n- Valid stu"
"dent ID required\n"
```

Read the three pieces. Which plan does the second piece describe? Whose price is `$14.99`, if all you have is that piece? Now cut the same file at each line that starts with `##`:

```
node -e 'const t=require("fs").readFileSync("plans.txt","utf8"); for (const s of t.split(/^(?=##
)/m)) console.log(JSON.stringify(s))'
```

```
"## Basic plan\nPrice: $9.99 per month\n- 2 GB of data\n\n"
"## Student plan\nPrice: $14.99 per month\n- 20 GB of data\n- Valid student ID required\n"
```

(`JSON.stringify` writes the line breaks as `\n` so that each piece fits on one line.) Two pieces, and each one is a whole plan: its name, its price and its features. Keep this picture in your head. It is the whole of milestone 2.

Exercise 3 · How alike are two sentences?

A computer cannot "see" that two sentences are alike. It turns each one into a list of numbers and compares the lists. The simplest list is a count of words. Save this as `similar.js`:

```
const words = s => s.toLowerCase().match(/[a-z0-9]+/g);
const vec = s => { const v = {}; for (const w of words(s)) v[w] = (v[w] ?? 0) + 1; return v; };
const dot = (a, b) => Object.keys(a).reduce((t, k) => t + a[k] * (b[k] ?? 0), 0);
const cosine = (a, b) => dot(a, b) / Math.sqrt(dot(a, a) * dot(b, b));
const q = vec(process.argv[2]);
for (const text of process.argv.slice(3)) console.log(cosine(q, vec(text)).toFixed(3), text);
```

Run it with a question, then some texts to compare it with:

```
node similar.js "student plan with 20 GB of data" "Student plan: 20 GB of data, student ID
required" "Basic plan: 2 GB of data" "Prepaid daily pass, pay only on the days you use it"
```

```
0.798 Student plan: 20 GB of data, student ID required
0.617 Basic plan: 2 GB of data
0.000 Prepaid daily pass, pay only on the days you use it
```

That number is the **cosine similarity**: 1 means the lists point the same way, and 0 means they have nothing in common. The Basic plan scores well too, because it shares "plan", "of" and "data". Write down one question you could ask where the *wrong* text would score highest. (Part 3 explains why this matters.)

Self-check (10 minutes)

Answer these in a sentence each, before you read Part 3 if you can. The answers are in Part 8. If you cannot answer one, you have found your first piece of work.

1. In one sentence, what is retrieval-augmented generation (RAG)?
2. Why can a model state a price that appears in no document?
3. Why is it better to cut a plan sheet at the headings than every 250 characters?
4. What does an embedding give you that the raw text does not?
5. What does a cosine similarity of 1 mean? Of 0?
6. Why should an answer come with a citation?
7. A model is shown no context and told nothing else. What does it do, and why?
8. What does it mean for a corpus to be stale?
9. A bot gives a wrong answer. Name one thing that points to a retrieval failure and one that points to a generation failure.
10. What does `npm run step -- 4` do?

Part 3 · The ideas

This part explains the ideas behind the lab in plain words. Each one ends with what it means for the work you will do. Five of the sections have a box called **In plain English**. If you are short of time, read just those.

3.1 Why a bot makes things up

A language model is very good at one job: continuing text in a way that sounds right. Ask it for the price of a plan and it writes the kind of sentence that follows such a question: a plan name, a price, a month. If it has never seen your plan sheet, it has no price to give. It writes a plausible one anyway. It does not know that it has nothing to say.

That is the whole hallucination problem. The bot is not lying. It is doing what it always does, with nothing to stand on.

> **In plain English.** A model with no document in front of it still has to say *something*. If you do not give it facts, it gives you fluent guesses. The fix is not a cleverer model. The fix is to hand it the right facts, tell it to use only those, and make it show where each fact came from.

In the lab: milestone 1 makes you see this with your own eyes, and keep the exact wrong answer, so that milestone 6 can prove it is gone.

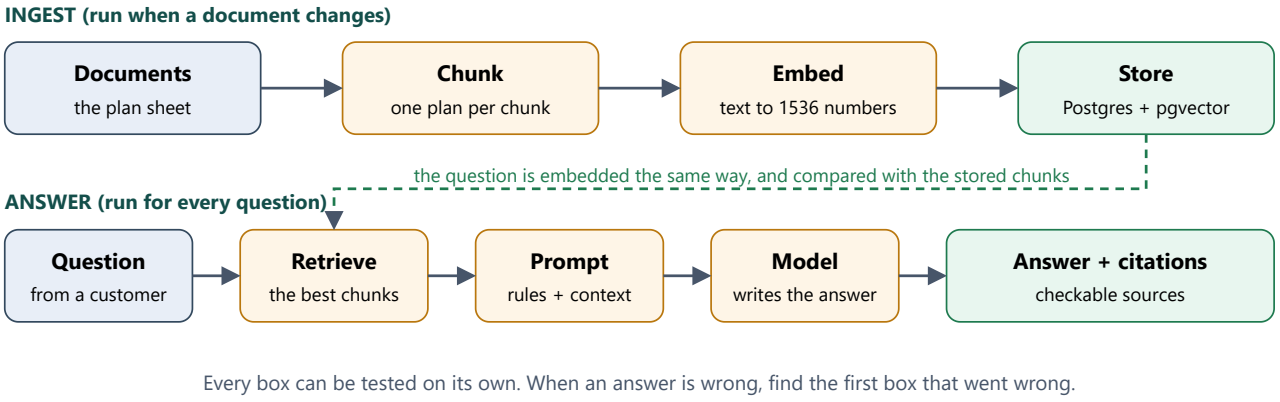
3.2 What "grounded" means

A **grounded** answer is one that is tied to a source you can check. The model answers from text that was put in front of it, and every claim points back to that text. If the source does not have the answer, the right output is "I don't have that", not a guess.

Two things are easy to miss:

- **Grounding is a property of the whole pipeline, not of a prompt.** The best prompt cannot ground an answer if the right text never reached the model. Most "the AI is wrong" complaints are really "the search found the wrong text" complaints.
- **A grounded answer can still be wrong.** If the source is out of date, the answer is faithful to it and still wrong. "Grounded" asks whether the answer is supported by the source. "Accurate" asks whether it is true. You need both, and they can come apart.

3.3 The pipeline at a glance



Every box can be tested on its own. When an answer is wrong, find the first box that went wrong.

The pipeline you build in the lab. The top row runs when a document changes, the bottom row runs for every question.

Stage	What it does	Milestone
Chunk	Cuts a document into pieces that each hold a complete fact	2
Embed	Turns each chunk (and later each question) into a list of numbers	3
Store	Keeps the chunk, its text and its numbers in a database	3
Retrieve	Finds the stored chunks closest to the question	4
Prompt	Puts the rules, the chunks and the question in front of the model	5
Model	Writes the answer, from the chunks only	5, 6
Cite	Returns the chunks the answer relied on, so a person can check	6

One sentence for the whole thing: **retrieval-augmented generation (RAG)** means finding the right text first, then asking the model to answer from that text.

3.4 Chunking: keep a fact with its context

A **chunk** is a piece of a document that is stored and searched as one unit. Cut it too small, and a fact loses the words that give it meaning. Cut it in the wrong place, and the fact is simply gone from the piece that mentions it.

Think of a price list. "$14.99 per month" means nothing on its own. It means something next to "Student plan". If a cut falls between the two, you now have a chunk with a price and no plan, and another with a plan and no price. No search, however clever, can put them back together.

Strategy	How it cuts	Good for	Watch out for
Fixed length	Every N characters	Plain prose, quick experiments	Cuts through names, prices and sentences
By structure	At headings, records, clauses	Price lists, policies, FAQs, anything with a shape	You must know the shape of the document

Strategy	How it cuts	Good for	Watch out for
By meaning	Where the topic changes	Long prose	Slower, and harder to test

For a document that already has structure, cut at the structure. A good chunk also has a **stable id** made from the thing it describes (`plan-unlimited-plus`), not from its position (`chunk-03`), so the id does not change when a plan is added.

> **In plain English.** Cut the document where the document itself changes, not after a number of characters. A chunk should be able to answer a question on its own: it must carry its own name and its own price.

In the lab: milestone 2. `npm run chunks` shows you what any chunker does to the sheet, and flags the damage.

3.5 Embeddings and similarity

An **embedding** is a list of numbers that stands for a piece of text. A model (or, in the lab, a small stand-in) reads the text and writes the list. Texts that are about similar things get lists that point in similar directions. A common size is **1536** numbers.

To compare a question with a chunk, you compare their lists. The usual measure is **cosine similarity**: it looks at the angle between the two lists. **1** means they point the same way, and **0** means they have nothing in common. In Part 2 you computed it by hand, with word counts instead of a learned embedding.

Retrieval is then simple: embed the question the same way, compare it with every chunk, and keep the top **k** (for example the top 5).

Three things to keep in mind:

- **"Similar" is not the same as "relevant."** A chunk can share many words with a question and still be about something else. In the warm-up, the Basic plan scored 0.617 for a question about the student plan, only because they share words like "plan", "of" and "data". Look at the first few results before you trust the numbers.
- **Embeddings find meaning, not logic.** They do not reliably handle "plans under $30, but not the family plan": the vector for that question is close to the family plan.
- **Use the same embedder for the chunks and for the question.** Lists from two different embedders cannot be compared.

> **In plain English.** An embedding is a fingerprint of a text's meaning, as a long list of numbers. Cosine similarity says how close two fingerprints are, from 0 to 1. Close fingerprints are a clue that two texts are about the same thing. They are a clue, not a proof.

In the lab: milestones 3 and 4. The lab's embedder is a small stand-in that needs no key and no network. It is real in the way that matters here: the numbers, the database column and the search are exactly what you would use in production.

3.6 Where the vectors live

You need somewhere to keep the chunks and their numbers, and a fast way to ask "which are closest to this one?" Three common choices:

Choice	What it is	Good for	Cost
In memory	A list in your program	A tiny corpus, an experiment	Nearly free, but it dies with the program
pgvector	An extension that gives Postgres a `vector` column and distance operators	Small to medium corpora, and teams that already run Postgres	One more extension on a database you have
A managed vector database	A service built for vectors (for example Azure AI Search)	Large corpora, high traffic, hybrid search	A new service to run and pay for

The lab uses **real Postgres with pgvector**, running inside Node, so there is nothing to install. In pgvector the operator `<=>` gives the cosine **distance** between two vectors. Distance is the opposite of similarity: 0 means identical.

Exact or approximate? Exact search compares the question with every row. That is instant for a dozen rows and slower for a million. An **index** such as ivfflat or HNSW takes shortcuts to be faster on big tables. A shortcut can skip rows, so an index is a trade: speed for a little accuracy, and it needs to be set up with care. The advice on a small corpus is simple. Fix retrieval quality first, and think about infrastructure second. A fancy database cannot rescue a corpus that was chunked badly.

3.7 Prompting for a grounded answer

The prompt is where the rules of your feature live. For a grounded bot there are three:

1. **Use only the context.** "Answer using ONLY the context below."
2. **Cite what you used.** "After each fact, write the chunk id in brackets."
3. **Say a fixed sentence when the answer is not there.** "If the answer is not in the context, respond exactly: ..."

Three more things to know:

- **An instruction is a promise, not a guarantee.** It only holds if the right context was put in front of the model. If retrieval missed the right chunk, the best you can hope for is a polite refusal, and the worst is an answer from somewhere else.
- **Test the "not found" path on purpose.** A bot that never says "I don't know" will, one day, invent something.
- **Treat the context as data, not as instructions.** A chunk is text from a document. If it says "ignore the rules above", that is part of the document, not a command. (Today's lab does not attack this. Later sessions do.)

> **In plain English.** Tell the model three things: use only what I gave you, show which piece you used, and say *this exact sentence* when you cannot find the answer. Then try a question you know the documents cannot answer, and check that it says the sentence.

In the lab: milestone 5. The model in the lab is a stand-in that reads your prompt literally, so a missing rule shows up at once.

3.8 Citations a person can check

A citation is only useful if someone can follow it. Three habits make the difference:

- **Return the chunk id and a snippet of its text** next to the answer, so that a person can verify the fact in seconds.
- **Cite only what was really retrieved.** Models sometimes name a source they were never given. Your code should drop any citation that does not match a chunk it actually passed in.
- **Treat an answer with no citation as a defect.** An answer with no source is not more trustworthy because it sounds sure of itself. It is just unverifiable, and unverifiable is how you got the invented price in the first place.

In the lab: milestone 6.

3.9 When an answer is wrong: find the layer

When the bot gives a wrong answer, there are two very different causes, and they need different fixes:

	Retrieval failure	Generation failure
What happened	The right chunk never reached the model	The right chunk was there, and the model said something wrong about it
How to tell	Look at the retrieved chunks (`npm run retrieve`). The right one is missing, or too low	The right chunk is in the list, but the answer does not match it
Usual fixes	Better chunks, a better question, more results, a rerank step	A clearer prompt, a smaller context, a check on the citations

If you cannot tell which layer failed, you do not know what to fix. You are just guessing at the guess.

The tool for this is a small set of **question and answer pairs**, written before you trust the feature, that you run after every change. That is what you wrote on Monday, and what milestone 7 builds. A set like this is the first, hand-made version of an evaluation harness. It must include questions the documents *cannot* answer, and it must be able to fail: a test that cannot fail proves nothing. On Wednesday 11 November (DD10) you will turn this idea into a proper, repeatable evaluation.

3.10 Keeping the corpus fresh

A grounded bot is only as current as its last **ingest**. If the price changes in the plan sheet and nobody re-ingests, the bot will keep quoting the old price, with a citation to a document that is out of date. Nothing in the answer tells the customer.

- **Detect it.** Compare when the source last changed with when it was last ingested.
- **Alert on it.** A check that fails loudly, and runs where changes ship (before a deploy), beats a note on someone's calendar.
- **Choose how to refresh.** For a small corpus, re-ingest everything. For a big one, re-embed only what changed.
- **Remove what is gone.** A plan that was deleted from the sheet must also leave the database, or the bot will keep selling it.

> **In plain English.** A perfectly grounded bot pointed at an out-of-date document is still wrong: it is just wrong with a source. Check whether the document changed since you last read it, and make the check fail loudly before anything ships.

In the lab: milestone 3 (ingest that can run again and removes old plans) and milestone 8 (the staleness check).

3.11 Reranking: a second, more careful look

Retrieval by vectors is fast and rough. It finds chunks that look like the question. A **rerank** step takes the handful it found (say 5) and puts them in a better order, by asking a more careful judge: "does this chunk directly answer what was asked?" The judge may be a model with a strict rubric, one call per candidate. It is slower and costs more, which is why you only use it on a few candidates and not on the whole corpus.

You will see one in the live demo, fixing a real wrong ranking. Reranking is an extra, and you can build the lab without it.

3.12 How this maps to Azure

In the lab	In your Azure estate
Real Postgres with pgvector, inside Node	Azure Database for PostgreSQL (flexible server) with the pgvector extension
A small stand-in embedder	An embedding model from Azure AI Foundry (or Azure OpenAI)
A rule-based stand-in model	A chat model from Azure AI Foundry, or the Claude API
`data/plan-docs.md`	A document store such as Azure Blob Storage, feeding an ingestion job
The retrieval you write in milestone 4	Azure AI Search: vector, keyword or hybrid search, with built-in chunking options
The fixtures you write in milestone 7	Evaluation tooling in Azure AI Foundry, for a first-pass groundedness check

Part 4 · How the lab works

4.1 Anatomy of a milestone card

Every milestone is a card in `LAB-GUIDE.md`, written like a ticket. Here is one for a made-up task that is **not** part of the lab, so you can see what each section is for.

Milestone X · Refuse an empty question *(an example, not a real milestone)*

Ticket FAQ-000 · aim for **8 minutes** · hints open at **+3, +5, +7** *The pace to aim for, and when each hint opens (counted from when you start the milestone).*

Why this matters. An empty question wastes a database search and returns a confusing answer. *The reason, so you can judge trade-offs.*

Your goal. `answer()` returns the not-found sentence for an empty question, without searching. *One outcome, in one sentence.*

You are given. `answer()` in `src/answer.js`, `config.notFound`, and `npm run ask`. *Everything you may use, so you never have to guess what exists.*

Interface contract. Return `{ answer, citations: [], sources: [], retrieved: [] }`. Do not change anything for a real question. *The exact shapes other code relies on. These are requirements, and they are not negotiable.*

Success criteria.

- **Must:** ☐ an empty question gets the not-found sentence ☐ a normal question still works ☐ `npm run step -- X` passes
- **Stretch:** ☐ also refuse a question of only spaces *How you will know you are done. "Must" is the finish line. "Stretch" is for when you are early.*

Where to start. What does `answer()` do today with an empty string? Where would a guard go? *Questions to settle before you type. They are not instructions.*

Verify yourself. `npm run step -- X` *The command that tells you which criteria are not met.*

The things a card does **not** contain: a prompt to paste, a solution, or the names of the traps. Finding those is the milestone.

4.2 Working mostly on your own

This lab is built so that you can work at your own pace, without anyone stopping the room after every step.

Block	Milestones	About how long	Then
A	1, 2, 3, 4	45 minutes	**Check-in 1:** the facilitator walks through the steps, the tests and the solutions for 1 to 4
B	5, 6, 7, 8	39 minutes	**Check-in 2:** the same, for 5 to 8

- **The pace on each card is a guide, not a deadline.** Some milestones will take you less, some more. A slow milestone costs you nothing, because syncing is free and the next milestone starts from known-good code.
- **You do not need permission to move on.** When `npm run step -- N` says PASSED, go on to the next milestone.
- **Pace marks in the chat.** At a few set times the facilitator posts a line such as "by now, aim to be starting milestone 3". It is there so that you can see how you are doing against the room. You do not need to reply.
- **At the check-ins** the facilitator shows the reference solutions and the reference briefs, and compares them with what you built. They may ask you to paste your `npm run progress` line in the chat (see 4.7).
- **If you finish a block early,** do the stretch goals on the cards. They are the best use of spare time. Or help a neighbour.

4.3 The hint ladder

Each milestone has **three hints** and then a **reveal**. They are meant to be taken in order, each only when you need it.

Level	What it gives you	Example (for a made-up milestone)
Hint 1: Nudge	Where to look, or the right question to ask	"What does the code do when the value is missing altogether?"
Hint 2: Approach	The shape of the solution, without the code	"Check the value before you use it, and stop early."
Hint 3: Near-solution	Almost the answer, enough to finish	"Compare the length with zero, and return before the search."
Reveal	The reference brief (a good prompt), the approach, and the common mistakes	Shown by the facilitator at the check-ins

How the clock works: it starts for a milestone the first time you run `npm run start -- 4`, `npm run step -- 4` or `npm run hint -- 4`. Each hint opens a few minutes after that. If you ask too early, the command tells you how long to wait:

```
Milestone 3 · Build the ingestion pipeline
Hint 1 of 3 opens in about 5 minutes (the clock for this milestone started 0 minutes ago).
Try the questions under "Where to start" on the card first. That is where most of the learning is.
If you really need it now:  npm run hint -- 3 --now   (it is noted as an early hint, and you can
say so in LAB-LOG.md)
```

That is a nudge, not a lock. `--now` always works, and the early hint is counted in your `npm run progress` table.

How to use hints well:

- **Try first.** A few minutes of being stuck is where most of the learning happens.
- **Take them with the command:** `npm run hint -- 4` gives the next one for milestone 4. `npm run hint -- 4 --again` re-reads the ones you have taken.

- **If you take one early, write it in your log.** That is not a penalty: it is how you will remember what you needed.
- **The reveal** is `npm run hint -- 4 --reveal`. It is meant for the check-ins, when the facilitator shows it. Reading it early is allowed, and honest logging is all that is asked.

4.4 Syncing: catching up for free

If you are not where you want to be, you do not fall behind:

```
npm run checkpoint -- 4
```

This replaces your code with the reference solution for milestone 4, so milestone 5 starts from known-good code. It **saves your own work first** in a folder called `.checkpoint-backup/`, so nothing is lost. It also clears the database, because the database is built from your code and the plan sheet: run `npm run ingest` afterwards to fill it again.

Using a checkpoint is not cheating. It is the reference solution, and the next milestone is built on it. What matters is that you understand it before you move on: read it, and write one line in your log about what it does differently from your version. You can sync at any time, not only at the check-ins.

4.5 Working with Claude Code

Claude Code is a very fast, very confident colleague. It reads the folder you open, follows the rules in `CLAUDE.md`, runs commands, and edits files. It is also sometimes wrong, and it will not always tell you. Your job in this lab is not to type; it is to **decide what "done" means and to check that it is**.

Writing a brief

Because there is no prompt to paste, the card's success criteria become the acceptance criteria in **your** brief. A good brief has five parts:

Part	What to say	Example
Context	What the code is, and what already exists	"This is a Node project. The answer function is in `src/answer.js`."
Task	One thing to do, in one sentence	"Make `answer()` return the not-found sentence for an empty question."
Constraints	What it must not touch or add	"Do not change `src/model.js` or `tests/baseline`. Add no dependencies."
Acceptance	The observable results that mean "done": file names, return shapes, what a run prints	"An empty question returns the not-found sentence with empty citations. A normal question behaves as before."
Verify	The commands that prove it	"Run `npm run step -- X` and `npm test`, and tell me the results."

A weak brief and a strong one, for the same task:

Weak	Strong
"Handle empty questions."	"In `src/answer.js`, if the question is empty, return `{ answer: config.notFound, citations: [], sources: [], retrieved: [] }` without calling `retrieve`. A normal question must behave exactly as now. Do not edit `tests/baseline`. Run `npm test` and report the result."
Claude has to guess where, what to return, and how you will judge it.	*Claude has a target it can check, and so do you.*

Four habits that matter more than any wording

1. **Ask for findings or a plan before any change.** "List what is wrong with this, and do not change anything yet" gives you something to check against reality before code moves.
2. **Read the diff.** Open the Source Control panel in VS Code, or run `git diff`. Look at which files it touched, and whether any are ones you did not expect.
3. **Run the check yourself.** "I ran the tests and they pass" is a claim until you have seen the output.
4. **Make Claude explain, then explain it back.** Several milestones ask for one sentence on why something was wrong. If you cannot write that sentence, you have not finished.

When Claude is confidently wrong

What you may see	What to do
It invents a file name or function that does not exist	Check against the card's interface contract. Say "use exactly this path and this name"
It changes files you told it to leave alone	Look at the diff. Revert them. Restate the constraint
It edits a test so the test passes, instead of fixing the code	Never accept a green check you did not earn. Ask why the test was wrong
It says "all tests pass" without running them	Ask it to run them and show the output
It does a lot more than you asked	Ask for the smallest change that meets the acceptance criteria
It adds a dependency, or calls a real AI service	The kit forbids it. The model in this lab is a local stand-in
It leaves a script running, and the next one says the database is "in use"	Stop the script (Ctrl+C), and ask it to close the database

Two safety rules

- **Never paste a real key, password or customer's data** into the chat or into Claude. Everything in the kit is made up. At work, nothing is.
- **The model is not the safety net.** A feature that is safe only because the model behaved is not safe. The checks you write, and the tests that prove them, are what protect the system.

4.6 Reading a failing check

When `npm run step -- N` fails it prints which success criteria are not met yet. Here is how to read one. This is a made-up failure:

```
MILESTONE X: Refuse an empty question

✓ a normal question still works (3.2ms)
✕ an empty question gets the not-found sentence (4.1ms)
ℹ tests 2
ℹ pass 1
ℹ fail 1

✕ failing tests:

✕ an empty question gets the not-found sentence (4.1ms)
  AssertionError [ERR_ASSERTION]: the bot searched for an empty question. It should say the not-
found sentence without searching.

STEP X not passing yet. Read the first failure above: it says which success criterion is not met.
Stuck? npm run hint -- X gives you the next hint when it opens. To catch up, npm run checkpoint --
X gives you the reference code.
```

- **The ✓ and ✕ lines** are the individual criteria. Fix them from the top.
- **`pass` and `fail`** are counts. You want `fail 0`.
- **The message after `AssertionError`** is the useful sentence. It says what was expected and what actually happened. (Milestone 1 is about finding facts yourself, so its check says which fact is wrong, but never what the right value is.)
- **The last two lines** tell you what to do next.

A failing check is information, not a verdict. Most are fixed by reading the message slowly.

4.7 The evidence log

`LAB-LOG.md` is yours. Two or three lines per milestone: what you tried, what you learned, which hints you took. Its purpose is to make the learning visible to you, and it is what you would show a colleague to explain how you got there. Nobody marks it.

Three prompts that make it useful:

- *What surprised me?*
- *What did Claude get wrong, and how did I notice?*
- *Could I explain this fix to someone else in one sentence?*

4.8 Asking for help, and the pulse

- **Say you are stuck in the chat.** Teaching assistants are watching it. "Stuck on 4, retrieval returns 1 row" gets a better answer than "it's broken".
- **The pulse.** At the check-ins the facilitator may ask for a pulse. Run `npm run progress` (it takes about a minute, because it runs your pipeline from scratch) and paste the last line:

```
PULSE M1[x] M2[x] M3[x] M4[ ] M5[ ] M6[ ] M7[ ] M8[ ] hints=3
```

`[x]` means that milestone's check passes, `[ ]` means not yet, and `hints` is how many hints you have taken in total. It tells the facilitator where the room is without anyone having to speak.

- **If several people hit the same problem,** a teaching assistant will post the fix in the chat. You may not be the only one.

4.9 Working with a buddy (optional)

If you like to work alongside someone, pair up at the start. Pairing in this lab means **comparing**, not splitting the work: each of you does the milestone, and at the check-in you explain to each other where your approaches differed. Explaining a fix to another person is the quickest way to find out whether you understand it.

Part 5 · The terminal handbook

Every command you will type on Wednesday, what it does, why we run it, and how to read what it prints. The lab VM runs Linux, and every command here is written for its shell (bash). The examples show real output from the lab kit, so what you see on the day will look the same, apart from things like dates and file paths.

Each entry has the same shape:

- **What it does**, in plain English
- **Why we run it** in this lab
- **How to read the output**
- **If it goes wrong**

5.1 How to read a command

A command is a program, then what to do, then the details:

```
npm run step -- 4
|   |   |    |  └─ an argument: the number 4
|   |   |    └──── "--": everything after this goes to the script, not to npm
|   |   └───────── the name of a script defined in package.json
|   └───────────── a subcommand: run a script
└───────────────── the program
```

Piece	Looks like	Meaning
Program	`npm`, `node`, `git`	The thing being run
Subcommand	`run`, `ci`, `diff`	What to ask the program to do
Flag	`-s`, `--k 3`, `--now`	Changes how it behaves. Some take a value
Argument	`4`, `"family data plan"`	What to act on
Comment	`# like this`	Ignored by the shell: it is there for you

Four things that catch people out:

1. **Quotes matter.** Inside **single quotes** (`'...'`) everything is literal. Inside **double quotes** (`"..."`) the shell replaces `$VARIABLES` with their values. A question with spaces needs quotes: `npm run ask -- "what's the price of the Unlimited Plus plan?"`. (If the question has an apostrophe, use double quotes outside, as here.)
2. **A backslash at the end of a line** (`\`) means "this command carries on on the next line". It must be the last character on the line, with no space after it.
3. **Copy-paste can change characters.** Word processors and chat tools turn straight quotes into curly ones, and two dashes into one long dash. Either breaks a command. If a pasted command fails mysteriously, retype the quotes and dashes.
4. **Case and spaces matter.** `npm run step -- 4` is not `npm run step --4`.

> **Tip: tab completion.** Press **Tab** while typing a file or folder name and the shell finishes it for you. Press it twice to see the options.

5.2 Moving around and looking at files

Command	What it does	Why we run it
`pwd`	Prints the folder you are in	To check you are in `faq-bot`, not one level up (the most common mistake)
`ls -A`	Lists the files in the folder, including hidden ones (names starting with a dot)	To see what is there. `-A` shows hidden files such as `.npmrc`
`cd dd6-lab/faq-bot`	Moves into a folder. `cd ..` moves up one level, `cd ~` goes home	To get to the right place before every npm command
`cat docs/findings.json`	Prints a file's contents	To read a small file without opening an editor
`head -n 20 file`	Prints the first 20 lines of a file or of a command's output	To look at the top of something long
`grep "Unlimited Plus" data/plan-docs.md`	Prints the lines of a file that contain some text	To search a document from the terminal
`touch data/plan-docs.md`	Sets a file's "last changed" time to now. It prints nothing, and not one word inside the file changes	In milestone 8, to pretend the plan sheet was edited, so that you can test your staleness check
Ctrl+C	Stops the program that is running in this terminal	To get out of anything that is stuck
`clear`	Clears the screen	To tidy up

If it goes wrong: `No such file or directory` almost always means you are in the wrong folder. Run `pwd`, then `ls`, then `cd` to where you meant to be.

5.3 Getting the kit

You will be given a link and a block of commands on the day. Here is what each one does.

```
cd ~
```

Go to your home folder, so the kit goes somewhere predictable.

```
unzip -q dd6-lab-kit.zip
```

Unpack the zip. `-q` means quiet: it does not list every file. If `unzip` is not installed, use `python3 -m zipfile -e dd6-lab-kit.zip .`, which does the same job.

```
cd dd6-lab/faq-bot
```

Move into the folder you will work in. **Only this folder is opened in VS Code.** The folders next to it (`hints`, `checks`, `checkpoints`) hold answers, and Claude Code reads everything in the folder you open.

```
ls
```

```
CLAUDE.md  LAB-GUIDE.md  LAB-LOG.md  data  docs  migrations  node_modules  package-lock.json
package.json  scripts  src  tests
```

```
npm ci
```

Installs the project's dependencies, exactly the versions in `package-lock.json`. `ci` stands for "clean install". It is a one-time step. This project has one dependency, PGlite, which is Postgres packaged to run inside Node.

```
added 1 package in 2s
```

> **`npm ci` or `npm install`?** `npm install` may pick newer versions and rewrite the lockfile. `npm ci` installs exactly what the lockfile says, and fails if it is inconsistent. Everyone in the room gets identical code, which is what a lab needs.

If it goes wrong: `ETIMEDOUT` or a proxy error means the VM cannot reach the npm registry: tell the facilitator, and do not fight it. `Unsupported engine` means Node is too old (`node --version` must say v20 or higher).

5.4 Node and npm

Command	What it does	Why we run it
`node --version`	Prints the Node version, for example `v24.14.1`	To check Node is new enough (v20 or higher)
`npm test`	Runs the project's tests	To confirm the starting point is healthy (`pass 18`), and to protect you from breaking things
`npm run <name>`	Runs a script named in `package.json`	Almost everything in the lab is a script: `ask`, `step`, `hint`, `ingest` …
`npm run help`	Prints every lab command with a one-line explanation	When you forget one

`npm test`

```
✓ the plan sheet has 12 plans, each with a price line (0.702ms)
✓ the made-up prices the stand-in model uses are not in the plan sheet (0.1385ms)
✓ the database starts in memory with pgvector working (1099.4861ms)
...
ℹ tests 18
ℹ pass 18
ℹ fail 0
```

How to read it: each `✓` is a test that passed and each `✕` one that failed. The last block is the summary: you want `fail 0`. These are the lab's own tests of the parts you are given (the embedder, the model, the database). They stay green all session. If one turns red, you changed something that you were asked to leave alone.

The double dash

```
npm run step -- 4
```

Without `--`, npm would try to interpret `4` itself. The `--` means "everything after this belongs to the script". Whenever you pass an option or argument to an npm script, put `--` before it. (`npm run step -- 4`, `npm run ask -- "a question"`, `npm run retrieve -- "a question" --k 3`.)

The `-s` flag

```
npm run -s ask -- "what does the Basic plan include?"
```

npm normally prints a few lines of its own before your script runs (the script name and the command). `-s` means *silent*: it removes them. It is handy when you want clean output, for example to paste an answer into a note.

5.5 Looking inside the database

The lab keeps its data in a real Postgres database, with the **pgvector** extension for numbers that stand for text. It runs inside Node and keeps its files in a hidden folder, `.data/`, so there is no server to start. You look inside it with one command.

`npm run sql -- "SELECT ..."`

What it does: runs one SQL statement against the lab database, and prints the rows as a table. **Why we run it:** to see for yourself what ingestion stored. It is your window into the database.

```
npm run sql -- "SELECT count(*) FROM plan_chunks"
```

```
count
-----
12

(1 row)
```

```
npm run sql -- "SELECT chunk_id, plan_name, length(text) AS chars FROM plan_chunks ORDER BY
chunk_id"
```

```
chunk_id                plan_name           chars
----------------------  ------------------  -----
plan-basic              Basic               112
plan-business-starter   Business Starter    149
plan-family-data-plan   Family Data Plan    201
...
(12 rows)
```

A column that holds an embedding has 1536 numbers, which is too many to read. Ask for its size, or for the start of it:

```
npm run sql -- "SELECT chunk_id, vector_dims(embedding) AS dims, left(embedding::text, 40) AS start
FROM plan_chunks ORDER BY chunk_id LIMIT 3"
```

```
chunk_id              dims  start
--------------------  ----  ----------------------------------------
plan-basic            1536  [0,0,0,0,0,-0.19518,0,0,0,0,0,0,0,0,0,0,
plan-business-starter  1536  [0,0,0,0,0,-0.163846,0,0,0,0,0,0,0,0,0,0
plan-family-data-plan  1536  [0,0,0,0,0,-0.264906,0,0,0,0,0,0,0,0,0,0
```

How to read it: the first line has the column names, then the rows, then how many rows there are. Mostly zeros is normal for this lab's embedder: it spreads a few words over 1536 slots.

One more you may want: `SELECT key, value FROM ingest_meta` (when you last ingested).

`EXPLAIN` : ask Postgres how it will run a query. Put the word `EXPLAIN` in front of any `SELECT` and it prints its plan instead of the rows. The line to look for is how it reads the table. This is the plan for "the five chunks closest to the first one", on a table with no index:

```
npm run sql -- "EXPLAIN SELECT chunk_id FROM plan_chunks ORDER BY embedding <=> (SELECT embedding
FROM plan_chunks LIMIT 1) LIMIT 5"

QUERY PLAN
------------------------------------------------------------------------
Limit (cost=25.17..25.18 rows=5 width=40)
 InitPlan 1
 -> Limit (cost=0.00..0.03 rows=1 width=32)
 -> Seq Scan on plan_chunks plan_chunks_1 (cost=0.00..15.20 rows=520 w
 -> Sort (cost=25.14..26.44 rows=520 width=40)
 Sort Key: ((plan_chunks.embedding <=> (InitPlan 1).col1))
 -> Seq Scan on plan_chunks (cost=0.00..16.50 rows=520 width=40)

(7 rows)
```

How to read it: `Seq Scan` means Postgres looked at **every row**, which is slow on millions of rows and always right. `Index Scan using <name>` means it took a **shortcut** through an index instead. If a query gives a different answer from what you expect, ask which of the two it used. (The numbers after `cost=` are the planner's own guesses. You can ignore them.)

If it goes wrong:

- `relation "plan_chunks" does not exist` : the table has not been created yet (milestone 3), or the database was cleared. Run `npm run ingest` .

- `The database folder is in use by another process`: another script of yours is still running. Wait for it, or stop it with Ctrl+C. The database can only be open in one process at a time.

5.6 The lab commands

`npm run ask -- "your question"`

What it does: asks the FAQ bot a question and shows the answer, its citations and its sources. **Why we run it:** it is the feature itself. You ask it at the start, to see the bug, and at the end, to see it fixed.

```
npm run ask -- "what's the price of the Unlimited Plus plan?"
```

```
Question : what's the price of the Unlimited Plus plan?
Answer   : The Unlimited Plus plan costs $64.99 per month.
Citations: none. Nothing here can be checked against the plan sheet.
```

How to read the output: the answer is what a customer would see. The `Citations:` line says which chunks the answer relies on. When it says `none`, nothing in the answer can be checked against the documents. `--json` prints the whole result as JSON.

`npm run chunks`

What it does: runs **your** `chunkPlans()` on the plan sheet, prints what it made, and says what is wrong with it. **Why we run it:** it is the quickest feedback for milestone 2. You never have to guess what your chunker did.

```
The plan sheet has 12 plans. chunkPlans() made 8 chunks.

#   chunkId                   planName              chars heading price
1   chunk-01                  unknown               250   yes     yes
2   chunk-02                  unknown               250   yes     yes
...

2 problem(s):
  - "Unlimited Plus": a chunk boundary cuts its heading in two (between chunk 2 and chunk 3)
  - expected one chunk per plan (12), got 8
```

How to read it: one row per chunk, and whether the chunk has a plan heading and a price line. Under the table, the problems. When every plan is in one chunk, it says so and exits with success. Add `-- --show` to print the text of every chunk.

`npm run ingest`

What it does: fills the database from the plan sheet. You write this script in milestone 3. **Why we run it:** after you change the sheet, the chunker or the way you store things, the database has to be rebuilt.

```
Ingested 12 chunks from /home/you/dd6-lab/faq-bot/data/plan-docs.md
```

(The message is whatever your script prints, so yours may differ.) Running it twice is meant to be safe.

`npm run retrieve -- "your question"`

What it does: shows which chunks **your** `retrieve()` returns for a question, best first. `--k 3` asks for 3 instead of 5. **Why we run it:** to look at retrieval with your own eyes, which is the only way to judge it.

```
npm run retrieve -- "is there a plan for tablets"
```

```
Question: is there a plan for tablets

rank  score   chunkId                   planName
1     0.371   plan-tablet-data-plan     Tablet Data Plan
2     0.130   plan-basic                Basic
3     0.113   plan-family-sms-bundle    Family SMS Bundle
4     0.088   plan-family-data-plan     Family Data Plan
5     0.000   plan-unlimited            Unlimited
```

How to read it: `score` is the cosine similarity, so a bigger number is a better match. Look at the first row: is it the plan the question is about? Then look at the others: a gap between the first score and the rest is a good sign.

`npm run reset-db`

What it does: deletes the database, so you can start again from nothing. **Why we run it:** after you change how you store things, an old database can hide the effect of your change. Then run `npm run ingest`.

`npm run start -- N`

What it does: starts the clock for milestone N and says when its hints open. **Why we run it:** it is optional. The first `step` or `hint` command for that milestone starts the clock for you.

```
Milestone 3 · Build the ingestion pipeline
The clock for this milestone just started.
Aim to finish in about 13 minutes. Hints open after 5, 9, 12 minutes (a nudge, not a lock: npm run
hint -- 3).
Open LAB-GUIDE.md at the card for this milestone, and read it before you type anything.
```

`npm run step -- N`

What it does: runs the check for milestone N against your work, and tells you which success criteria are not met yet. **Why we run it:** it is how you know you are done, without asking anyone. It checks what your code **does**, not what you named things. Read Part 4.6 for how to read a failure.

When it passes, the end looks like this:

```
STEP 2 PASSED
Milestone 2 is done. Write two lines in LAB-LOG.md, then go on to the next milestone.
```

`npm run progress`

What it does: runs the check for every milestone and prints a table, and one line to paste in the chat. It reads your work and never changes it. It takes about a minute. **Why we run it:** at the check-ins, the facilitator may ask for it, so that they can see where the room is.

```
  [x]  1  Reproduce the hallucination            done  (7 pass, 0 fail)  hints 0
  [x]  2  Chunk the plan sheet                   done  (9 pass, 0 fail)  hints 1
  [ ]  3  Build the ingestion pipeline           not yet  (2 pass, 4 fail)  hints 0
  ...

2 of 8 milestones done, 1 hints taken.

PASTE THIS IN THE CHAT:
PULSE M1[x] M2[x] M3[ ] M4[ ] M5[ ] M6[ ] M7[ ] M8[ ] hints=1
```

How to read it: `[x]` is a milestone that passes. The `hints` column says how many hints you took, and "(1 early)" marks hints you took before they opened.

`npm run hint -- N`

What it does: gives you the next hint for milestone N, one level at a time, once it has opened. If you ask too early, it tells you how long to wait. `--now` takes it anyway (it is counted as early), `--again` re-reads the hints you have taken, and `--reveal` shows the reference solution (see Part 4.3). With no number, it lists how many hints you have taken for each milestone. **Why we run it:** to get unstuck without giving up the learning.

`npm run checkpoint -- N`

What it does: replaces your work with the reference solution for milestone N, saves your own work first, and clears the database. **Why we run it:** to catch up for free, so that the next milestone starts from known-good code.

```
Your code now matches the reference solution for milestone 3.
Your previous work was saved in .checkpoint-backup/2026-10-28T03-12-44-102Z
The database was cleared too. If this milestone is 3 or later, run  npm run ingest  to fill it
again.
Run `npm test` to confirm, then carry on with the next milestone.
```

`npm run fixtures`

What it does: runs the script **you** write in milestone 7. It asks the bot every question in `tests/fixtures/qa.json`, compares each answer with what you expected, and exits with a failure code if even one does not match. It is not there until you add it to `package.json`. **Why we run it:** so that you find out a change broke an answer before a customer does.

```
PASS  what's the price of the Unlimited Plus plan?
PASS  what's included in the basic plan
PASS  how many GB does the student saver include
...
PASS  do you offer international roaming in Japan?

12 of 12 passed
```

(Yours will have your own questions. The wording of each line is whatever your script prints.) A `FAIL` line should say **why**, and in particular whether retrieval or generation was to blame.

`npm run deploy`

What it does: a pretend deploy. It prints `DEPLOYED` and does nothing else. What matters is what runs **before** it: npm runs a script called `predeploy` first, if there is one, and **stops** if it fails. In milestone 8 you make `predeploy` your staleness check. **Why we run it:** to prove that a stale corpus cannot ship. Run it when everything is fresh, then after `touch data/plan-docs.md`.

```
npm run deploy                      <- the sheet has not changed since the last ingest
OK: the plan sheet has not changed since the last ingest (2026-10-28T08:15:00.000Z).
DEPLOYED: the FAQ bot is live (pretend). If the plan sheet had changed since the last ingest, it
would be serving old prices.

touch data/plan-docs.md
npm run deploy                      <- now the sheet is newer than the last ingest
STALE: the plan sheet changed at 2026-10-28T08:21:15.985Z, after the last ingest at 2026-10-
28T08:15:00.000Z. Run: npm run ingest
```

On the second run there is no `DEPLOYED` line: npm stopped at the failed check. Then `npm run ingest` and `npm run deploy` go ahead again.

`npm run doctor`

What it does: checks Node, the install, the plan sheet, the folder layout, and that the database starts with pgvector working. **Why we run it:** it is the quickest way to find out why nothing works. It ends with `Ready.` when everything is fine.

`npm run baseline`

What it does: commits the starting point with git, so that `git diff` shows only what **you** changed. **Why we run it:** so that you can see exactly which files Claude touched. Run it once, at the start.

`npm run help`

Prints every lab command with a one-line explanation. It is the same list as the cheat sheet at the back of this handbook.

5.7 The git commands you will use

You do not need to know git to do the lab. You need three commands, all of which only **read**.

Command	What it does
`git status --short`	Lists the files that changed since the baseline. `M` means modified, `??` means new
`git diff --stat`	One line per changed file, with how many lines were added and removed
`git diff`	Shows exactly what changed

```
 M src/chunk.js
?? migrations/001_plan_chunks.sql
```

```
diff --git a/src/chunk.js b/src/chunk.js
@@ -3,9 +3,12 @@
-const SIZE = 250; // characters per chunk
+export function chunkPlans(markdown) {
+  return markdown.split(/^(?=## )/m)...
```

How to read a diff. Lines starting with `-` (and shown in red in an editor) were **removed**. Lines starting with `+` (green) were **added**. Lines with neither are unchanged context. `@@ -3,9 +3,12 @@` says where in the file this is. Look at **which files** changed first: a file you did not expect to see is the thing to question. VS Code shows the same information in the Source Control panel.

5.8 Environment variables

An environment variable is a named setting a program can read. Putting `NAME=value` **in front of one command** sets it for that command only:

```
FAQ_DATA_DIR=/tmp/scratch-db node scripts/ingest.js      # ingest into a scratch database, not
yours
```

Variable	Read by	What it does
`FAQ_DATA_DIR`	everything that opens the database	Where the database folder is (default `.data/pg`)
`PLAN_DOCS`	`config.planDocs`	The path of the plan sheet (default `data/plan-docs.md`)
`ROAMING_DOCS`	`config.roamingDocs`	The path of the roaming policy (default `data/roaming-policy.md`)

You will rarely type these yourself. They matter because the lab's checks use them to run **your** code against a scratch database and a scratch copy of the plan sheet, so that a check can change a price or delete a plan without touching your files. That is why your code must read the paths from `config`, and never type them in.

5.9 When something goes wrong

What you see	Most likely cause	What to do
`No such file or directory`	You are in the wrong folder	`pwd`, then `cd` to `~/dd6-lab/faq-bot`

What you see	Most likely cause	What to do
`Could not read package.json` or `Missing script`	You are in the wrong folder (often one level up), so npm cannot find the project	`pwd`, then `cd ~/dd6-lab/faq-bot`
`unzip: command not found`	The VM has no `unzip`	`python3 -m zipfile -e dd6-lab-kit.zip .`
`npm ERR! ... ETIMEDOUT` or `E404`	The VM cannot reach the npm registry	Tell the facilitator. Pair up in the meantime
`Unsupported engine` or a `SyntaxError` on a normal import	Node is older than v20	`node --version`, and ask for help
`The database folder is in use by another process`	Another script of yours has the database open	Wait for it, or Ctrl+C it. Ask Claude to close the database at the end of a script
`relation "plan_chunks" does not exist`	The table is not created yet, or the database was cleared	`npm run ingest` (and check your migration is in `migrations/`)
`retrieve() is not written yet`	You have not done milestone 4 yet	Do it, or `npm run checkpoint -- 4`
My change to the plan sheet is not in the answers	The database still holds the old text	`npm run ingest` again
My code change does nothing	You are running a different file, or the database holds old data	Check the path in the error, then `npm run reset-db` and `npm run ingest`
`npm run step` says `Missing .../checks/...`	The folders are not side by side	Keep `faq-bot`, `checks`, `checkpoints` and `hints` together in `dd6-lab`
The check passes on my machine but a colleague's fails	You may be on different checkpoints	`npm run progress` on both, and compare
Claude changed files you did not expect	It did more than asked	`git diff --stat`, revert what is wrong, restate the constraint
The terminal seems frozen	A program is still running in it	Ctrl+C
You are not sure what a command does		`npm run help`, or this handbook

Part 6 · The kit, mapped

You get the kit at the start of the session. This is what is in it, so that nothing is a surprise. Everything in it is **made up**: the company, the plans, the prices and the roaming policy.

The folders

```
dd6-lab/
├── README.md              setup, the commands, what is real and what stands in
├── faq-bot/               <- the only folder you open in VS Code
│   ├── LAB-GUIDE.md       your milestone cards
│   ├── LAB-LOG.md         your evidence log
│   ├── CLAUDE.md          the rules Claude Code reads first
│   ├── data/              the plan sheet and the roaming policy (made up)
│   ├── src/               the code: some you write, some you are given
│   ├── scripts/           the commands: some you write, some you are given
│   ├── migrations/        your SQL files (the table definitions)
│   ├── docs/              your written results
│   └── tests/             the lab's tests, and (later) your question and answer pairs
├── hints/                 the hints and the reference briefs  (do not open)
├── checks/                what `npm run step` runs              (do not open)
└── checkpoints/           the reference solutions               (do not open)
```

Why only `faq-bot/` is opened: Claude Code reads everything in the folder you open, and the three folders next to it hold the answers.

The code you write, and the code you are given

You write (milestone)	You are given (leave alone)
`src/chunk.js` (2; it starts as a starter that is not good enough)	`src/config.js` : the paths, `defaultK` and the not-found sentence
`scripts/ingest.js` and a migration in `migrations/` (3)	`src/db.js` : opens the database, runs your migrations
`src/retrieve.js` (4)	`src/lib/embed.js` : the stand-in embedder
`src/prompt.js` (5)	`src/model.js` : the stand-in model
`src/answer.js` (6; it starts as the buggy bot)	`data/` : the plan sheet and the roaming policy
`tests/fixtures/qa.json` and `scripts/fixtures.js` (7)	`scripts/ask.mjs` , `chunks.mjs` , `retrieve.mjs` , `sql.mjs` and the lab scripts
`scripts/check-staleness.js` (8)	`tests/baseline/` : the lab's own tests

The plan sheet

`data/plan-docs.md` is the only source of plan names, prices and features. It has a title and an intro, then one section for each of 12 plans:

```
## Student Saver
Price: $14.99 per month
- 20 GB of data
- Unlimited calls
- Valid student ID required
- ID is checked again every year
```

A section starts at a line that begins with `##` , and runs until the next one. The first lines of the file (the title and the intro) belong to no plan. The homework adds a second document, `data/roaming-policy.md` , which is shaped a little differently.

The database

Table	What is in it	Created in
`plan_chunks`	One row per chunk: `chunk_id` , `plan_name` , `text` , and `embedding` (1536 numbers)	Milestone 3
`ingest_meta`	Small facts about the corpus. One row, `last_ingested_at` , says when you last ingested	Milestone 3
`schema_migrations`	Which of your `.sql` files have already run (created by the lab)	Automatically

The database lives in a hidden folder, `.data/` . It is rebuilt from your code and the plan sheet, which is why `npm run reset-db` is safe.

What is real and what stands in

In this kit	Stands in for	Notes
PGlite with pgvector (`src/db.js`)	Azure Database for PostgreSQL (flexible server) with pgvector	Real Postgres and real pgvector: the same SQL runs in production.
The embedder (`src/lib/embed.js`)	An embeddings API (Azure OpenAI or another)	Hashes words into 1536 numbers. A real embedder captures meaning, not just shared words.
The model (`src/model.js`)	The Claude API (or a model in Azure AI Foundry)	A rule-based stand-in: the same input gives the same output, with no network. It reads your prompt literally.
`data/*.md`	Azure Blob Storage, a CMS or a plan database	A made-up plan sheet and roaming policy.

How the stand-in model behaves (so that you can predict it): if the system prompt does not say to use only the context, it answers from "general knowledge", which means it makes things up and cites nothing. If it does, it answers only from the chunks it is given, takes the not-found sentence from the quoted sentence in your prompt, and cites only if your prompt asks. It also answers about the plan whose name it finds in the question: for a made-up plan that starts like a real one ("Unlimited Ultra") it answers for the real one, and no wording of your prompt changes that. That is a finding for your log, not a mistake in your prompt. The comment at the top of `src/model.js` says all this in detail: read it before milestone 5.

Part 7 · Glossary

Term	Plain-English meaning
Chunk	A piece of a document that is stored and searched as one unit. In the lab, one plan
Citation	A pointer from a claim in an answer to the chunk it came from, so a person can check it
Corpus	The set of documents the bot may answer from
Cosine distance / similarity	Two ways to say how close two lists of numbers are. Similarity is 1 when they point the same way and 0 when they share nothing. Distance is the opposite: 0 means identical. pgvector's `<=>` gives the distance
Embedding	A list of numbers (here 1536) that stands for a piece of text. Similar texts get similar lists
Exit code	The number a program leaves behind when it stops. 0 means success. Anything else means failure. `npm` stops a chain of steps when one fails
Fixture	A fixed example used in a test. A question with the answer you expect is a fixture
Grounded	Tied to a source you can check. A grounded answer is supported by the text it was given
Hallucination	A fluent answer that no source supports. The model is continuing text, not looking anything up
Idempotent	Safe to run again: running it twice leaves things as running it once would
Index (vector)	A shortcut for finding the closest vectors faster than comparing with every row, for example ivfflat or HNSW. It trades a little accuracy for speed
Ingest	Reading the documents, chunking, embedding and storing them, so that they can be searched
Migration	A `.sql` file that changes the database's structure. It runs once, in file-name order
Not-found sentence	The exact words the bot says when the documents do not have the answer
pgvector	An extension for Postgres that adds a `vector` column type and distance operators
Prompt (system and user)	The text the model receives. The system part holds the rules. The user part holds the context and the question
RAG	Retrieval-augmented generation: find the right text first, then ask the model to answer from it
Rerank	Re-ordering the few results that retrieval found, using a more careful (slower) judge
Retrieval	Finding the stored chunks that best match a question
Stale	Out of date. A corpus is stale when a document changed after it was last ingested
Top-k	The k best matches. With k = 5, the five closest chunks
Upsert	"Insert, or update if it is already there." In Postgres: `INSERT ... ON CONFLICT ... DO UPDATE`
Vector	A list of numbers. In this lab, the embedding of a chunk or a question

Term	Plain-English meaning
Milestone / checkpoint / hint	A milestone is one ticket. A checkpoint is the reference solution for one, which you can sync to. A hint is a nudge that opens on a clock

Part 8 · Answers, reading and the cheat sheet

Answers: the warm-up

Exercise 2. The second piece (`ent plan ... Valid stu`) starts in the middle of the word "Student". All it holds is a price and a feature list with no plan name, so `$14.99` could belong to anything. A chunk cut by length separates a fact from its name. Cutting at `##` keeps each plan whole, and that is what a good chunker does.

Exercise 3. Any question where the wrong text shares more words than the right one will do. For example: "*how much is the student plan*" is closest to a text that says "*how much is the plan*" than to the one that says "*Student plan: 20 GB of data, $14.99*". Word counts measure overlap, not meaning, which is why you always look at the first few results.

Answers: the self-check

1. **RAG** is: find the right text first, then ask the model to answer from that text.
2. A model continues text in a way that sounds right. With no document in front of it, it produces a plausible price. It cannot tell that it has nothing to say.
3. A fixed cut falls wherever the count runs out, even between a plan's name and its price. Cutting at the headings keeps the name, the price and the features together, so a chunk can answer a question by itself.
4. A way to compare text by meaning, with arithmetic. Texts that are about the same thing get lists of numbers that point the same way, so "find the closest" becomes a maths problem.
5. **1** means the two lists point in exactly the same direction (as alike as they can be). **0** means they share nothing.
6. So that a person can check the claim against the source in seconds. An answer with no citation is unverifiable, however sure it sounds.
7. It still has to answer, so it answers from what it learned in training: fluent, confident, and not tied to any of your documents. That is a hallucination.
8. A document changed after the corpus was last ingested, so the bot is answering from an old version of it.
9. Retrieval failure: the right chunk is not among the retrieved chunks (or is far down the list). Generation failure: the right chunk is in the list, but the answer says something it does not support.
10. It runs the check for milestone 4 against your work, and prints which of the success criteria are not met yet. When all are met it prints `STEP 4 PASSED`.

Frequently asked questions

Do I have to use Claude Code? No. Brief it, do it by hand, or do both. The checks only look at what your code does.

What if I finish early? Do the stretch goals on the cards. They are the best use of spare time, and some are harder than the main task.

What if I fall behind? Use the hints, then `npm run checkpoint -- N`. Syncing is free. A slow milestone costs you nothing, because the next one starts from known-good code.

Is taking a hint early cheating? No. Write it in your log. The hints are there to be used. Taking one without trying first is how you miss the learning.

Why is the model a stand-in, and not the real Claude API? A lab for forty people needs the same answer every time, with no key and no network. The stand-in reads your prompt literally, so a missing rule shows up at once. The shape of the code is the same as for a real model: messages in, text out.

Is this real Postgres? Yes. It is PGlite, which is Postgres compiled to run inside Node, with the real pgvector extension. The SQL you write works unchanged on Azure Database for PostgreSQL.

Why is the embedder a stand-in too? For the same reason. A real embedding model needs a key or a large download. The stand-in turns words into 1536 numbers in a repeatable way, so that you can see everything that matters: the column, the search, the scores. A real model is better at meaning and about equal at everything else.

I asked about a plan that does not exist and the bot gave me the price of a real one. Is my prompt wrong? Probably not. If the made-up name starts like a real plan ("Unlimited Ultra"), the stand-in model answers for the real one, whatever the prompt says. Put it in your log. A real deployment would add a check in code (does the question name a plan that is not in the sheet?). It is a good example of why a prompt is a promise and not a guarantee.

Why does the check for milestone 4 still fail after I repaired my own database? The check builds its own fresh database from your `migrations/` folder every time. A repair made by hand to your own database does not reach it. Put the repair in `migrations/`.

Will the check-ins be recorded? Ask your coordinator. The notes in this handbook are the same either way.

Where do I put my evaluation questions from Monday? In `tests/fixtures/qa.json`, in milestone 7. You will need to turn each into a question with an expected citation, or a not-found.

What is the capstone connection? The pipeline you build here, with its chunking, ingest, retrieval, prompt, citations and tests, is the starting point the capstone team extends. Keep your `LAB-LOG.md`.

Reading list

Short, and all worth the time. Checked on 26 October 2026.

- **pgvector** on GitHub: the README covers the distance operators and the two index types, and what to watch for when you create one.
- **"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"** (Lewis and others, 2020): the paper that gave RAG its name. The first two pages are enough.
- **Anthropic's documentation on reducing hallucinations**: the same three habits as Part 3.7, with more examples.
- **Azure AI Search**: the documentation on vector search and hybrid search.
- **OWASP Top 10 for LLM Applications**: the section on prompt injection, which is where "treat the context as data" comes from.

Apply it at work: ten questions for any grounded AI feature

1. What are the documents, and who owns them? Who tells us when one changes?
2. How are they cut into chunks, and could a fact be separated from its context?
3. Does every answer say where it came from? Can a person open the source in one click?
4. What does the bot say when the documents do not have the answer? Is that tested?
5. When an answer is wrong, can we tell whether retrieval or generation failed?
6. What happens when the best match is not the right one? Is there a rerank step, or at least a score floor?
7. How do we know the corpus is fresh? What stops a stale one shipping?
8. What is in the test set, and could it fail? Does it include questions that must be refused?
9. Who is allowed to see which documents? Does retrieval check that, or only the endpoint in front of it?
10. What does one answer cost in time and money, for the retrieval and the model together?

One-page cheat sheet

I want to...	Command
Ask the bot	`npm run ask -- "question"`
See what my chunker makes	`npm run chunks`
Fill the database	`npm run ingest`
Look inside the database	`npm run sql -- "SELECT ..."`
See what retrieval returns	`npm run retrieve -- "question"` (`--k 3`)
Start again from an empty database	`npm run reset-db`, then `npm run ingest`
Check a milestone	`npm run step -- N`
Check everything and get the chat line	`npm run progress`
Get a hint	`npm run hint -- N` (`--now` takes it early)
Catch up for free	`npm run checkpoint -- N`, then `npm run ingest`
Run my question and answer pairs (milestone 7)	`npm run fixtures`
Pretend to deploy (it refuses when the corpus is stale)	`npm run deploy`
Make the sheet look newer without changing it (milestone 8)	`touch data/plan-docs.md`
Run the lab's own tests	`npm test`
Find out why nothing works	`npm run doctor`

The five parts of a brief: context, task, constraints, acceptance, verify.

The three rules of a grounded prompt: use only the context, cite what you used, say the exact not-found sentence.

The pipeline: chunk, embed, store, retrieve, prompt, model, cite.

Remember:

- **Never** cut a document where it does not change.
- **Similar** is not the same as **relevant**.
- Look at the retrieved chunks before you blame the model.
- A test that cannot fail proves nothing.
- A grounded bot on a stale document is still wrong.